Fullbond

A HANDS-ON CLAUDE CODE PROJECT · NO CODING NEEDED · BUILT FROM THE FULLBOND WEBSITE BUILD PLAN

# Fullbond Email Studio Lab

You will set up an email marketing studio for **Fullbond** (fullbond.com.au) and run it with Claude Code. Claude plans the launch campaign, writes the 5-email welcome series that turns free-checklist sign-ups into Melbourne Renter Toolkit buyers, personalizes it for each group of renters, and sends it, safely, to a practice email server. Along the way you use every Claude Code feature on the CCAR-F exam.

`15 steps` `about 30–60 min each` `no real emails are sent` `no Resend account needed`

## Before you start

1. Install **Node.js 22 or newer** from nodejs.org. Check with `node --version`.
2. Install **Git**. Check with `git --version`. On Windows, install Git for Windows, because Claude Code uses its Git Bash.
3. Install **Claude Code**: `npm install -g @anthropic-ai/claude-code`, then sign in when you first run `claude`.
4. Make your studio folder and open Claude Code in it:
   `mkdir fullbond-email-studio` · `cd fullbond-email-studio` · `git init` · `claude`
5. Copy your **Fullbond — Website Build Plan.pdf** into the folder as `notes/fullbond-build-plan.pdf` (create the `notes` folder first). Several prompts tell Claude to read it.

HOW EVERY STEP WORKS

| 1 · BUILD | 2 · FRESH | 3 · CHECK | 4 · DONE |
|---|---|---|---|
| **Paste the build prompt** | **Start a new session** | **Paste the check prompt** | **Answer its questions** |
| In Claude Code, in your studio folder. | Type `/exit`, then run `claude` again. | A fresh Claude tests your work. It didn't build it, so it's honest. | Then tick Done on the step. |

# Your studio at the end

```
fullbond-email-studio/
├── CLAUDE.md               the studio's rules
├── brand/voice.md          how Fullbond sounds
├── leads/
│   ├── leads.csv           24 practice renters
│   └── CLAUDE.md           rules for this folder only
├── campaigns/welcome-series/
│   ├── plan.md             the campaign plan
│   ├── emails/             5 welcome emails
│   └── groups/             a version per renter group
├── notes/
│   └── fullbond-build-plan.pdf
├── .claude/
│   ├── rules/emails.md     rules for every email file
│   ├── skills/personalize/
│   └── hooks/send-guard.mjs
├── .mcp.json               connection to Resend (practice)
└── scripts/review.mjs      automatic review
```

**Claude will ask permission** before it writes inside `.claude/` or to `.mcp.json`. That's normal. Read what it wants to do, then approve.

**Your brand, your answers.** This lab uses your real Fullbond details. In Step 3 Claude interviews you, so answer in your own words.

# Fullbond at a glance

| What | Details (from the build plan) |
|---|---|
| Business | Fullbond, fullbond.com.au, contact hello@fullbond.com.au |
| Product | Melbourne Renter Toolkit: Move-In Kit $12, Move-Out Kit $15, Complete Toolkit $29 (best value) |
| Free offer (lead magnet) | The free **10 Bond Mistakes** checklist |
| Launch offer | Code **LAUNCH10**, 10% off, first 2 weeks |
| Promise | Instant download · Made for Victorian renters · 14-day money-back promise |
| Sending email | Resend: practice mode in this lab, your own verified domain (fullbond.com.au) when you go live |
| Campaign skills | Anthropic's marketing plugin: `/marketing:campaign-plan`, `/marketing:email-sequence`, `/marketing:brand-review`, `/marketing:draft-content` and more, plus your own `/personalize` skill |
| Must-haves in every email | Fullbond name, contact address, unsubscribe link (Spam Act 2003), "General information, not legal advice" |

# The welcome series you'll build

From Step 6 of the build plan. Five automatic emails after someone downloads the free checklist.

| Send | Subject line | What it says |
|---|---|---|
| Straight away | Your free bond checklist is here | Download link + one quick tip |
| Day 2 | The photo mistake that costs renters the most | How to take move-in photos that count as evidence |
| Day 4 | What's inside the Fullbond toolkit | Preview images, who it helps, link to buy |

| | | |
|---|---|---|
| Day 6 | Moving soon? Here's 10% off | Launch code, money-back promise, Buy button |
| Day 9 | Last reminder: your code ends tonight | Short and friendly, one button |

THE THREE RENTER GROUPS

| Group | Who they are | What they worry about |
|---|---|---|
| `moving-in` | Just signed a lease, about to get the keys | Proving the condition on day one (photos, condition report) |
| `moving-out` | Leaving in the next 6 weeks | Cleaning, final inspection, replying to a bond claim |
| `first-timers` | Renting for the first time, often students or new arrivals | Not knowing the rules or the words ("rental provider", "bond") |

# Campaign skills you'll use

Anthropic's official marketing plugin (installed in Step 2) gives you ready-made campaign skills. You run them by typing their name in Claude Code. Your own `/personalize` skill (Step 8) joins them.

| Skill | What it does for Fullbond | Where you use it |
|---|---|---|
| `/marketing:campaign-plan` | Turns the launch idea into a full plan: goal, audience, messages, calendar, metrics, risks | Step 4 |
| `/marketing:email-sequence` | Writes the 5-email welcome series from the plan | Step 6 |
| `/marketing:brand-review` | Checks every email against Fullbond's brand voice and email rules | Step 7 |
| `/personalize` | Your own skill: one version of each email per renter group | Step 8 |
| `/marketing:draft-content` | Drafts launch posts, guides and other content in Fullbond's voice | Bonus 1 below |
| `/marketing:competitive-brief` | Compares Fullbond with other bond and renter-help offers | Bonus 2 below |
| `/marketing:seo-audit` | Finds what renters search for, to plan your 3 launch guides | Bonus 3 below |
| `/marketing:performance-report` | Turns launch numbers into a short report with next steps | Bonus 4 below |

Do the bonus prompts any time after Step 3, once brand/voice.md exists. Each one saves its result to campaigns/ or notes/ so the next step can use it.

**BONUS 1** Launch posts, after Step 6

```
/marketing:draft-content Launch week posts for Fullbond on Instagram, TikTok and Facebook.

Use brand/voice.md for our voice and campaigns/welcome-series/plan.md for the key messages.
Write one "Fullbond is live" post per platform, each pointing people to the free 10 Bond
Mistakes checklist at https://fullbond.com.au/free-checklist. Also write one short post for
Melbourne renter groups that is helpful first and mentions Fullbond only at the end. No
guarantees, no legal advice, no fake urgency.
```

```
Save them to campaigns/welcome-series/social/launch-posts.md, then commit.
```

**BONUS 2** Know the market, before Step 4

```
/marketing:competitive-brief Fullbond sells a $29 Melbourne Renter Toolkit (checklists and email templates for getting your bond back) and a free 10 Bond Mistakes checklist.

Compare us with the free help renters already use, such as Consumer Affairs Victoria guides and tenant advice services, and with any paid bond or end-of-lease checklists you can find. Show what each offers, what it costs and how it talks to renters. Then give me three ways Fullbond can stand out honestly, without claiming to replace legal advice.

Save it to notes/competitive-brief.md, then commit.
```

**BONUS 3** Plan your guides, after Step 4

```
/marketing:seo-audit fullbond.com.au is about to launch. Plan the first 3 guides from our build plan: "How to get your bond back in Victoria", "Move-out cleaning checklist" and "What to do on move-in day".

For each guide, suggest the search phrases Melbourne renters are likely to use, a page title under 60 characters, a meta description under 155 characters, and the headings to cover. Each guide should end by offering the free checklist.

Save it to notes/seo-plan.md, then commit.
```

**BONUS 4** After launch week

```
/marketing:performance-report Fullbond's first week after launch.

Here are my numbers: [sign-ups], [emails opened], [clicks to the toolkit], [toolkit sales], [LAUNCH10 uses]. Compare them with the goal in campaigns/welcome-series/plan.md. Tell me which welcome email is working best and which needs work, and suggest one change to test next week.

Save it to campaigns/welcome-series/report-week-1.md, then commit.
```

PART 1 · STEPS 1–8

# Plan and write

Set up the studio, give it Fullbond's voice, plan the welcome campaign and write the emails. Nothing gets sent in this part.

1

Real-world habit 2.4

## Look before you build

Search for skills other people made, and decide which ones to trust.

> **In plain words:** a skill is like an app for Claude. Before you build one, look in the app store. Download counts can be faked, so also check who made it.

**BUILD** Paste in Claude Code

```
I'm setting up an email marketing studio in this folder for Fullbond, a small Australian
online business that sells a $29 downloadable toolkit to Melbourne renters and grows its list
with a free checklist. Before I build anything myself, I want to find skills other people have
already made.

1. Search the skills directory at skills.sh by running these one at a time:
   npx skills find email marketing
   npx skills find email sequence
   npx skills find welcome series
   npx skills find resend
   npx skills find campaign plan
2. For the most promising results, find out who publishes each one and how many stars its
GitHub repository has. Look out for official publishers such as anthropics and resend.
3. Write notes/skills-i-chose.md in plain English with three short sections:
   - Use these: the two or three sources you recommend for planning campaigns, writing
campaign emails and sending email, and why. Name the campaign skills each one gives me (for
example campaign-plan or email-sequence).
   - Avoid these: two results you would not trust, and why.
   - How I decided: the three things I should check before installing any skill.

Don't install anything yet.
```

**CHECK** New session: `/exit`, then `claude`, then paste

```
Check my work in notes/skills-i-chose.md. Don't change any files. Test it, don't just read it:

1. For each skill the note recommends, confirm it still exists today: look at its page on
skills.sh or its folder in the GitHub repository. Tell me if any has been renamed, merged or
removed.
2. Run one of the searches again (for example npx skills find email marketing) and confirm the
install numbers in the note are roughly right.
3. Is anything recommended from a publisher whose repository has very few GitHub stars?
```

```
4. Does the note recommend at least one trustworthy campaign-planning skill, and list which
campaign skills it provides?

Give me a short table: Check | OK or Problem | What you found.
Then ask me two quick questions, one at a time, to see if I understand why install counts
alone aren't enough.
```

**YOU'LL KNOW IT WORKED WHEN**

- The note picks official or well-known sources, like Anthropic's marketing plugin and Resend's own skills.
- It warns you off a result with thousands of installs but only a handful of GitHub stars.
- It names the campaign skills you'll use later: `campaign-plan`, `email-sequence` and `brand-review` from Anthropic's marketing plugin.

**EXAM NOTE**

The exam asks the same question about MCP servers: for a standard need, use an existing community server first, and build your own only for work that is special to your team.

2

3.1

## Set up your studio

Install a plugin for this project only, and write the studio's CLAUDE.md.

> **In plain words:** CLAUDE.md is the studio's notice board. Claude reads it every time it starts work here. A plugin installed for this project is like equipment fixed to this studio's wall: anyone who joins the studio gets it too.

**BUILD** Paste in Claude Code

```
Let's set up the Fullbond email studio in this folder.

1. Install Anthropic's official marketing plugin for this project only, by running these two
commands:
   claude plugin marketplace add anthropics/knowledge-work-plugins --scope project
   claude plugin install marketing@knowledge-work-plugins --scope project
2. Create four folders: brand, leads, campaigns and notes (notes may already exist; keep
what's in it).
3. Write a CLAUDE.md for the studio in plain English. Include:
   - What the studio is for: planning and writing email campaigns for Fullbond
(fullbond.com.au), which sells the Melbourne Renter Toolkit (Move-In Kit $12, Move-Out Kit
$15, Complete Toolkit $29) and grows its list with the free "10 Bond Mistakes" checklist.
Email is sent through Resend; checkout is Payhip. Campaigns are planned and written with the
marketing plugin's campaign skills.
   - That notes/fullbond-build-plan.pdf is the source of truth for products, prices, pages and
the welcome series.
   - A one-line description of each folder.
   - These rules: write in plain Australian English; never send email to real people; every
email must include the Fullbond name, a contact address and an unsubscribe link (Spam Act
2003); Fullbond gives general information, not legal advice; ask me before deleting anything.
4. Create a .gitignore that ignores .claude/settings.local.json and system files like
.DS_Store.
5. Commit everything with a clear message.

Then tell me what I have to do so the plugin loads.
```

### Then

1. Type `/exit` and run `claude` again. Plugins load when a session starts.
2. Accept the prompts to trust the folder and the plugin.
3. Type `/marketing:` and you should see the plugin's commands appear.

**CHECK** New session, then paste

```
Check that my studio is set up correctly. Don't change any files. Test it, don't just read it:

1. Is the marketing plugin turned on for this project only? Compare .claude/settings.json with
what `claude plugin list` says about its scope.
2. Can you use the plugin here right now? List the plugin commands you can see whose names
include campaign, email or brand.
3. If a classmate clones this repo, will they get the same plugin? Explain why, using what is
actually in the repo.
4. Is .claude/settings.local.json ignored by git? Prove it with `git check-ignore -v
.claude/settings.local.json`.
5. Without opening CLAUDE.md, tell me the three Fullbond prices and the name of the free
checklist. Which file did you get them from?

Give me a short table: Check | OK or Problem | What you saw.
Then ask me two quick questions, one at a time, about the difference between files the whole
team shares and files that are only mine.
```

**YOU'LL KNOW IT WORKED WHEN**

- `claude plugin list` shows **marketing** with **Scope: project**.
- You can see `/marketing:campaign-plan`, `/marketing:email-sequence` and `/marketing:brand-review`.
- `git check-ignore` names your `.gitignore` line.
- The fresh session knows the prices and the "10 Bond Mistakes" checklist without opening a file.

**EXAM NOTE**

Team standards go at project level: CLAUDE.md in the repo, `.claude/settings.json`. Teammates get them when they clone. Your personal files (`~/.claude/CLAUDE.md`, `settings.local.json`) never reach anyone else. If a new teammate isn't getting the instructions, they sit at user level.

3

3.1 3.5

## Fullbond's brand voice

Let Claude interview you, then keep the voice in its own file that CLAUDE.md imports.

> **In plain words:** a tailor takes your measurements before cutting the cloth. Here Claude asks first, then writes. The `@brand/voice.md` line is a note pinned to the notice board that says "also read the brand book".

**BUILD** Paste in Claude Code

```
Interview me about the Fullbond brand so you can write our brand voice guide. First read
notes/fullbond-build-plan.pdf, especially "What customer friendly means for Fullbond" and the
design rules, so your questions build on it instead of repeating it. Then ask one question at
a time and wait for my answer. Ask at most six questions.

Cover at least: who our renters are, how we want them to feel after reading an email, words
that sound like us, words and tactics we avoid (the plan says: no fake reviews, no pressure
tactics, plain words like "rental provider" and "bond", not legal jargon), claims we must
never make, and how we sign off.

When you have enough, write brand/voice.md with five parts: our voice in three words, words we
use, words we avoid, claims we must never make, and how we sign off. "Claims we must never
make" must include: guaranteeing anyone gets their full bond back, presenting anything as
legal advice, fake urgency or scarcity, and reviews or testimonials we don't really have.

Then add the line @brand/voice.md to CLAUDE.md so it loads in every session.
```

### Then

1. Answer each question in your own words. This is your real brand, so take your time.
2. When the file is written, ask Claude to commit it.

**CHECK** New session, then paste

```
Test my brand voice setup. Don't change any files.

1. Without opening any file first, write one subject line for the Day 4 email, "What's inside
the Fullbond toolkit". Then tell me which brand rules you followed and which file they came
from. (This shows whether brand/voice.md loads by itself.)
2. Now write a subject line that says "Guaranteed: get your full bond back or we'll fight your
landlord for you." If our rules forbid that, say which rules, and give me a safe version
instead.
3. Run git status and tell me whether the brand files are committed.
```

```
Give me a short table: Test | Pass or Fail | What happened.
Then ask me two quick questions, one at a time, about why the brand rules live in their own
file that CLAUDE.md imports.
```

**YOU'LL KNOW IT WORKED WHEN**

- The fresh session follows your voice **without opening any file**, and names `brand/voice.md` as the source.
- It refuses the "guaranteed" line, names the rules (no guarantees, no legal advice, "rental provider" not "landlord"), and offers a safe subject line.

**EXAM NOTE**

`@import` keeps CLAUDE.md small: each project imports only the standards it needs. The interview pattern is for unfamiliar work: Claude asks before it builds and raises things you didn't think of.

**Exam answer: to see which memory files are loaded, use /memory. In today's product, /context shows it.**

4

3.2

## Plan the campaign

Use a ready-made skill from the plugin to turn the welcome series into a full plan.

> **In plain words:** `/marketing:campaign-plan` is a planning form a marketing team already tested. You fill in the blanks and it builds the structure.

**BUILD** Paste in Claude Code (change the numbers in [brackets] to your own targets)

```
/marketing:campaign-plan Launch Fullbond with an automated 5-email welcome series for people who download the free 10 Bond Mistakes checklist.

Goal: [100] Melbourne Renter Toolkit sales from checklist sign-ups in the first 4 weeks after launch.
Audience: everyone who signs up for the free checklist, in three groups: moving-in (just signed a lease), moving-out (leaving within 6 weeks) and first-timers (renting for the first time).
Offer: Complete Toolkit $29, with Move-In Kit $12 and Move-Out Kit $15. Launch code LAUNCH10 gives 10% off for the first 2 weeks. 14-day money-back promise. Instant download.
Emails: use the 5-email welcome series in notes/fullbond-build-plan.pdf (Step 6): straight away, Day 2, Day 4, Day 6, Day 9. Also include the buyer thank-you email 7 days after purchase asking for a short review.
Timeline: launch week starts Monday 7 December 2026. The LAUNCH10 code runs for the first 2 weeks.
Budget: small. Email plus free social posts and renter groups, no paid ads.

Save the finished plan to campaigns/welcome-series/plan.md, then commit it.
```

**CHECK** New session, then paste

```
Check the campaign plan in campaigns/welcome-series/plan.md. Don't change any files. Test it, don't just read it:

1. Is there one goal with a number and a deadline?
2. Does every email in the calendar have a send time (day after sign-up), an audience group and a purpose? Do they match the 5 emails in notes/fullbond-build-plan.pdf?
3. Search the plan for any word or claim that brand/voice.md says to avoid. Quote every one you find, with its line.
4. Are the prices, the LAUNCH10 code and the 14-day promise exactly as in the build plan?
5. Does Monday 7 December 2026 really fall on a Monday, and when exactly does a 2-week LAUNCH10 window end? Work it out, don't guess.

Give me a short table: Test | Pass or Fail | What you found.
Then ask me two quick questions, one at a time, about why we used an existing planning skill instead of writing our own prompt.
```

**YOU'LL KNOW IT WORKED WHEN**

- `plan.md` has about ten sections: goal, audience, messages, channels, calendar, metrics and risks.
- Prices and the code match the build plan exactly, and no "guaranteed" promises crept in.
- If the check finds a date mistake, ask Claude to fix it.

**EXAM NOTE**

Type `/marketing:campaign-plan` with nothing after it and look at the hint. That hint comes from the skill's `argument-hint` setting, which tells you what to type when you run the skill without arguments.

5

3.5 3.1

## Practice leads

Use two example rows to get the exact format, and give one folder its own rules.

> **In plain words:** two example rows work like a photo of the dish instead of a description of it. `leads/CLAUDE.md` is a sign on one room's door: Claude reads it when it walks into that room.

**BUILD** Paste in Claude Code

```
Make a list of practice leads for the welcome series.

Create leads/leads.csv with 24 made-up people who downloaded the free 10 Bond Mistakes
checklist, 8 in each group: moving-in, moving-out and first-timers. Use real Melbourne
suburbs. Use exactly these columns, and copy the style of these two example rows exactly:

first_name,last_name,email,group,suburb,joined_on,consent
Priya,Nair,priya.nair@example.com,moving-in,Brunswick,2026-11-02,checklist form
Liam,O'Connor,liam.oconnor@example.com,moving-out,Footscray,2026-11-04,checklist form

Every email address must end in @example.com. That domain is reserved and can never receive
mail, so no real person is ever emailed.

Then create leads/CLAUDE.md with rules for this folder only: these are made-up practice
people; never add real people or real email addresses; every row needs a consent value
(Australia's Spam Act needs consent); never change the @example.com addresses; and a "Groups"
section with one line describing each group.

Commit when you're done.
```

**CHECK** New session, then paste

```
Test my practice leads. Don't change any files.

1. Count the people in leads/leads.csv and how many are in each group. There should be 24,
with 8 per group.
2. Do all email addresses end in @example.com? Are any repeated?
3. Does every row have a suburb, a joined_on date and a consent value?
4. Now open leads/leads.csv and pretend I asked: "Add my housemate Jess Tran,
jess.tran@gmail.com, as moving-out." Before writing anything, tell me if any rule stops you,
and quote it. Don't actually add her.

Give me a short table: Test | Pass or Fail | What you found.
Then ask me two quick questions, one at a time, about why some rules belong to one folder
only.
```

**YOU'LL KNOW IT WORKED WHEN**

- 24 people, 8 per group, every address at `@example.com`.
- Claude refuses to add the Gmail address and quotes `leads/CLAUDE.md`, even though it never opened that file. It loaded on its own when Claude opened a file in `leads/`.

**EXAM NOTE**

When prose instructions get misread, give 2–3 input/output examples, not more prose. A CLAUDE.md inside a subfolder loads when Claude works on files in that folder.

6

3.3

## Email rule and welcome series

Write a rule that attaches itself to every email file, then write the five emails.

> **In plain words:** a path rule is a sticky note that attaches itself to every email file, in any campaign folder. CLAUDE.md is read for everything. This note shows up only when Claude works on an email.

**BUILD 1 OF 2** Paste in Claude Code, then approve the edit

```
Create a rule that applies to every email file in any campaign, and only to email files.

Make .claude/rules/emails.md. It must start with exactly this front matter:

---
paths:
  - "campaigns/**/emails/*.md"
---

Under it, write the rule in plain English. Each email file must:
- start with a line "Subject: ..." of 50 characters or fewer, then a line "Preview: ..." of 90
characters or fewer
- greet the reader with {{first_name}}
- be short enough to read on a phone: 150 words or fewer in the body
- have exactly one main link. Use [Get the toolkit — $29](https://fullbond.com.au/toolkit) for
sales emails, or [Download your checklist](https://fullbond.com.au/free-checklist) for the
first email
- never promise a full bond back, never give legal advice, and never invent urgency; a
deadline may only be mentioned if it is real (LAUNCH10 ends after the first 2 weeks)
- end with our sign-off from brand/voice.md, then this three-line footer as plain text, not
inside a code block:
  You're getting this because you downloaded the free 10 Bond Mistakes checklist from
Fullbond.
  Fullbond · ABN {{abn}} · hello@fullbond.com.au · {{company_address}} · Unsubscribe:
{{unsubscribe_url}}
  Fullbond provides general information, not legal advice.

Commit when you're done.
```

### Then

1. Start a new session ( `/exit` , then `claude` ) so the new rule is picked up.

**BUILD 2 OF 2** Paste in the new session

```
/marketing:email-sequence Welcome series for people who download Fullbond's free 10 Bond Mistakes checklist.

Use campaigns/welcome-series/plan.md for the goal, timing and key messages, the welcome-series table in notes/fullbond-build-plan.pdf for each email's subject and purpose, and brand/voice.md for our voice. Write 5 emails: straight away, Day 2, Day 4, Day 6 and Day 9.

Save each email as its own file in campaigns/welcome-series/emails/, named 01-short-name.md, 02-short-name.md, and so on. Then commit.
```

**CHECK** New session, then paste

```
Test the emails in campaigns/welcome-series/emails/. Don't change any files. Measure, don't eyeball: count characters, words and links with a command where you can.

For each email file, check:
1. The Subject line is 50 characters or fewer, and the Preview line is 90 or fewer. Give the real counts.
2. It greets the reader with {{first_name}}.
3. It has exactly one link, and it is one of the two allowed links.
4. The body is 150 words or fewer.
5. It ends with a sign-off from brand/voice.md and the exact three-line footer from .claude/rules/emails.md, as plain text.
6. It uses no word or claim that brand/voice.md says to avoid, and no "guarantee" or legal-advice wording.

Give me one table with a row per email and a column per check (Pass or Fail, with the number or quote when it fails).
Then ask me two quick questions, one at a time, about why this rule uses a paths pattern instead of living in CLAUDE.md.
```

**YOU'LL KNOW IT WORKED WHEN**

- Five email files, each starting with `Subject:` and `Preview:`, with real character counts under the limits.
- One link per email, the footer as plain text, and the "not legal advice" line in every one.
- Without the rule, the plugin tends to write three subject options and a notes section in each file. With the rule, every email follows the studio's format.

**EXAM NOTE**

For conventions that apply to files spread across folders, use `.claude/rules/` with `paths:` patterns. A subfolder CLAUDE.md covers one folder only. One big CLAUDE.md with headings relies on Claude guessing. A skill only works when someone runs it. Use `/context` to see which rules loaded.

7

3.6 3.5 2.5

## Fresh-eyes review

Review in a new session, fix related problems in one message, then confirm with search.

> **In plain words:** when you proofread your own letter, you see what you meant to write. A new session is a second reader who didn't write it.

**BUILD 1 OF 2** New session, then paste

```
/marketing:brand-review campaigns/welcome-series/emails/

Review all five emails against brand/voice.md and .claude/rules/emails.md. Also check them
against the "customer friendly" rules in notes/fullbond-build-plan.pdf: honest, low risk to
buy, helpful even if they don't buy, no pressure tactics. List every problem by severity
(high, medium or low), with the file, the line and a suggested fix. Don't change any files
yet.
```

**BUILD 2 OF 2** Same session, after you read the review

```
Fix all the problems from your review in one message, keeping all five emails consistent with
each other (same prices, same code, same promise wording). If a fix means changing a rule file
too, change it. Leave anything only I can confirm, like what the toolkit page says or whether
our refund promise has conditions, and list it for me. Then commit.
```

**CHECK** New session, then paste

```
Check that the review fixes worked. Don't change any files.

1. Use Glob to list every email file in every campaign.
2. Use Grep to search all of them for three backticks in a row, for "guarantee", "landlord",
"legal advice" (outside the footer), and for every word that brand/voice.md says to avoid.
There should be no matches.
3. Use Grep to confirm "$29", "LAUNCH10" and "14-day" are written the same way everywhere they
appear.
4. Run git show --stat HEAD and tell me which files the last commit changed.
5. Confirm the footer is plain text in every email.

Give me a short table: Test | Pass or Fail | What you saw.
Then ask me two quick questions, one at a time: why the review happened in a new session, and
why all the fixes went in one message.
```

**YOU'LL KNOW IT WORKED WHEN**

- The review lists problems by severity. Expect small but real ones, like a promise to "reply within the hour" when the site promises 1 business day.
- After the fixes, Grep finds nothing, and the last commit touched only email files and maybe the rule.

**EXAM NOTE**

The session that wrote something keeps its own reasoning and rarely questions it, so review in a fresh, independent session. Fixes that affect each other go in one detailed message; separate fixes can go one at a time. Glob finds files by name; Grep finds text inside files.

8

3.2

## Your own skill: personalize

Build a skill with the three settings the exam asks about, and use it.

> **In plain words:** a skill is a recipe card. Claude always sees the card's title, and reads the whole card only when it cooks that dish. `context: fork` sends a helper to another kitchen to do the messy work; only the finished dish comes back to you.

**BUILD 1 OF 2** Paste in Claude Code, then approve the edit

```
Create my own skill for this project at .claude/skills/personalize/SKILL.md.

What it does: it takes a campaign folder and writes one version of each email for each lead
group in leads/leads.csv (moving-in, moving-out and first-timers). It reads the group
descriptions in leads/CLAUDE.md. Each version keeps the same structure, subject, link, price,
code and footer, but changes the opening line and one example so it fits that group (for
example, move-in photos for moving-in, the final inspection for moving-out, and plain
explanations of "bond" and "rental provider" for first-timers). It saves each version to
<campaign folder>/groups/<group>/emails/ with the same file name. When it finishes, it replies
with a short summary only: how many files it wrote, plus one example opening line per group.

Use exactly this front matter:
---
name: personalize
description: Write a version of each campaign email for every lead group in leads/leads.csv.
Use when a campaign's emails are final and you want one copy per group.
argument-hint: "[campaign folder]"
context: fork
allowed-tools: Read, Glob, Grep, Write
---

If no campaign folder is given, it should stop and list the campaign folders that exist.
Commit when you're done.
```

### Then

1. Start a new session so the skill is picked up.

**BUILD 2 OF 2** Paste in the new session

```
/personalize campaigns/welcome-series
```

**Then**

1. Ask Claude: *"Commit the new files with a clear message."*

**CHECK** New session, then paste

```
Test my personalize skill and what it made. Don't change any files.

1. Count the email files under campaigns/welcome-series/groups/. There should be one per email
per group: 5 emails times 3 groups is 15.
2. Compare the original email 01 with each group's version using diff. Only the opening line
and one example should differ. Tell me if anything else changed, especially the price, the
code or the link.
3. Check that every version still follows .claude/rules/emails.md: subject 50 characters or
fewer, preview 90 or fewer, body 150 words or fewer, one link, and the footer as plain text.
Measure, don't eyeball.
4. Does any version use a word or claim that brand/voice.md forbids? Quote it if so.

Give me a short table: Test | Pass or Fail | What you found.
Then ask me two quick questions, one at a time: what "context: fork" changed about how the
skill ran, and what "allowed-tools" did.
```

**YOU'LL KNOW IT WORKED WHEN**

- 15 new files. Only the opening line and one sentence differ from the original.
- Your main chat shows only a short summary, not the work itself.
- The email rule applied to the new files too, because they also sit in an `emails/` folder.

**EXAM NOTE**

`context: fork` runs the skill in its own context, so long output stays out of your chat. `argument-hint` tells you what to type when you run it with no arguments. A personal copy goes in `~/.claude/skills/` under a different name, so it doesn't clash with the team's.

**Exam answer: allowed-tools restricts which tools the skill can use. In the product it pre-approves the listed tools, and other tools still ask permission. You may see that happen. On the exam, answer "restricts".**

Not real: a `.claude/config.json` file with a commands list.

PART 2 · STEPS 9–11

# Send safely

Connect a real email service through MCP, but point it at a practice server on your own computer. Then add a guard that makes sure no email can reach the wrong person.

**Why practice mode?** Resend has a free practice server you can run on your own computer, so you can practise sending without any risk. When you're ready to go live, you point the same connection at your real Resend account and your verified fullbond.com.au domain.

9

2.4

## Connect Resend in practice mode

Connect an email service through MCP, with settings that are safe by default.

> **In plain words:** an MCP server is a phone line from Claude to another service. In practice mode that line goes to a pretend post office on your own computer, so nothing leaves your machine.

### Before you start

1. Open a **second terminal** window and run: `npx emulate@0.12.3 start --service resend`
2. Wait until it says **Ready** and shows `http://localhost:4000`. Leave that window open for steps 9 to 11.

BUILD Paste in Claude Code, then approve the edit

```
Connect our studio to Resend, the email service, in practice mode.

Create .mcp.json in the project root with exactly this content:

{
  "mcpServers": {
    "resend": {
      "command": "npx",
      "args": ["-y", "resend-mcp@2.27.1"],
      "env": {
        "RESEND_API_KEY": "${RESEND_API_KEY:-re_practice}",
        "RESEND_BASE_URL": "${RESEND_BASE_URL:-http://localhost:4000}",
        "SENDER_EMAIL_ADDRESS": "${SENDER_EMAIL_ADDRESS:-onboarding@resend.dev}"
      }
    }
```

```
  }
}

Then update the rule in CLAUDE.md that says never to send email to real people. Keep that
rule, and add that sending through Resend is allowed only to Resend's test addresses (ending
in @resend.dev) and to my own email address, and that a guard will enforce this in the next
step.

Commit. Then explain in two plain sentences what happens when I set nothing, and what I would
set to send for real.
```

### Then

1. Start a new session and approve the **resend** server when Claude Code asks.
2. Type `/mcp`. You should see resend connected.

**CHECK** New session, then paste

```
Test my Resend connection. Don't change any files.

1. Use the resend MCP tools to send one email to delivered@resend.dev with the subject
"Fullbond connection test" and a one-line body. Did it go through? Give me the email id.
2. Use the resend tools to list sent emails, and confirm that one shows up as delivered.
3. Search the whole repo, including git history, for anything that looks like a real Resend
key: "re_" followed by a long string, other than re_practice. There should be none.
4. Is .mcp.json committed, so a teammate who clones the repo gets the same connection?

Give me a short table: Test | Pass or Fail | What you saw.
Then ask me two quick questions, one at a time: why the key is written as ${RESEND_API_KEY:-
re_practice} instead of the real key, and what happens if I forget to set anything.
```

**YOU'LL KNOW IT WORKED WHEN**

- The test email gets an id and shows as delivered.
- No real key appears anywhere in the repo or its history.

**IF SOMETHING GOES WRONG**

- **Connection refused:** the practice server isn't running. Start it again in your second terminal.
- **Pending approval:** start `claude` and approve the resend server.

**EXAM NOTE**

The team's servers go in `.mcp.json`, which is committed. Your personal or experimental servers go in `~/.claude.json`. Credentials use `${VAR}` expansion, so no secret ever lands in the repo. `${VAR:-default}` means "use my setting if I have one, otherwise this safe value". If Claude keeps picking Grep over an MCP tool, the tool's description is too thin.

10

1.5

## The safety guard

Make a rule Claude cannot break, using a hook.

> **In plain words:** CLAUDE.md asks Claude to behave. A hook is a guard at the post office door who checks every letter before it leaves. Claude can't talk its way past the guard.

**BUILD** Replace `[your email]` before you press Enter

```
Add a safety guard so Claude can never send email to the wrong people, even by mistake.
CLAUDE.md asks nicely. This guard makes sure. Under Australia's Spam Act we must only email
people who agreed to it, so this matters.

1. Write .claude/hooks/send-guard.mjs, a small Node script for a PreToolUse hook. It reads the
tool call as JSON from standard input and looks at tool_name and tool_input:
   - For send-email and send-batch-emails: every address in to, cc and bcc must end in
@resend.dev or match my own email, which comes from the MY_EMAIL environment variable. If any
address doesn't, print a short reason that names the address and exit with code 2 to block the
call.
   - For send-broadcast, create-automation, send-inbox-draft and send-new-inbox-email: always
block, with a short reason, because they can reach whole lists of people.
   - For anything else: exit 0 to allow it.
2. Register it in .claude/settings.json as a PreToolUse hook with the matcher mcp__resend__.*
and this command:
   node "$CLAUDE_PROJECT_DIR/.claude/hooks/send-guard.mjs"
   Keep everything that is already in that file.
3. Put my own email in .claude/settings.local.json like this: "env": { "MY_EMAIL": "[your
email]" }. That file is only mine and is not committed.
4. Commit the hook and .claude/settings.json.
```

### Tip

Use the same email you would sign up to Resend with. If you ever send for real, Resend's free sandbox can only deliver to that address. You could use hello@fullbond.com.au once your business email is set up.

**CHECK** New session, then paste

```
Test the send guard for real. Don't change any files.

1. Use the resend send-email tool to send email 01 from campaigns/welcome-series/emails/ to
Priya, the first person in leads/leads.csv, at her address. This should be blocked. Quote the
block message word for word.
2. Send the same email to delivered@resend.dev. This should go through.
3. Send it to my own email, the MY_EMAIL value in .claude/settings.local.json. This should go
```

```
through too.
4. Run git check-ignore -v .claude/settings.local.json to prove my email address is not
committed.

Don't try to get around a block. Give me a short table: Test | Pass or Fail | What happened.
Then ask me two quick questions, one at a time: what the difference is between the CLAUDE.md
rule and this guard, and what exit code 2 means for a hook.
```

**YOU'LL KNOW IT WORKED WHEN**

- The send to Priya is **blocked**, with the guard's own message naming her address.
- The sends to `delivered@resend.dev` and to your email go through.

**EXAM NOTE**

**Exam answer: the guide names only PostToolUse, for cleaning up tool results. It describes blocking as "tool call interception" without naming the hook.**

In the product, PreToolUse is what blocks, with exit code 2 or a deny decision, and PostToolUse can't block. On the exam, match the behaviour described.

11

Real-world skill

## Practice launch

Send each group its first welcome email, safely, and check that it was delivered.

**BUILD** Paste in Claude Code

```
Let's do a practice launch of the welcome series.

For each lead group, send that group's version of email 01 (from campaigns/welcome-series/groups/<group>/emails/) to delivered@resend.dev. Start each subject with the group name in square brackets, like [moving-out]. Replace {{first_name}} with "there", and leave the other placeholders as they are.

Then list the emails you sent and show me a table: group, subject, status.
```

**CHECK** New session, then paste

```
Check my practice launch. Don't send anything new and don't change any files.

1. Use the resend tools to list sent emails. Find the three whose subjects start with [moving-in], [moving-out] and [first-timers].
2. For each one, look up its delivery status. Is it delivered?
3. Confirm that every email in the list went only to an @resend.dev address or to my own email, the MY_EMAIL value in .claude/settings.local.json. List any that didn't.
4. Compare each sent subject with the subject line in that group's email 01 file. Do they match, apart from the [group] tag?

Give me a short table: Test | Pass or Fail | What you saw.
Then ask me two quick questions, one at a time: what would change if I switched from practice mode to my real Resend account, and why the guard still matters then.
```

**YOU'LL KNOW IT WORKED WHEN**

- Three emails tagged `[moving-in]`, `[moving-out]` and `[first-timers]`, all delivered, all to test addresses.

▼ **Optional: send one real email to yourself through Resend**

1. Sign up for a free account at resend.com with the same email as your `MY_EMAIL`.
2. In Resend, create an API key. Never paste it into a file or into the chat.
3. Close Claude Code. In your terminal, set the key and the real address for this window only.
   macOS or Linux: `export RESEND_API_KEY=re_your_key` and `export RESEND_BASE_URL=https://api.resend.com`
   Windows PowerShell: `$env:RESEND_API_KEY="re_your_key"` and `$env:RESEND_BASE_URL="https://api.resend.com"`
4. Run `claude` in that same window and ask: *"Send the moving-out version of email 01 to my own email, the MY_EMAIL value."*
5. Check your inbox. Without your own domain, Resend only delivers to the email you signed up with, and the guard blocks everyone else anyway.

PART 3 · STEPS 12–14

# Work like a team

Pick work up across days, try two ideas side by side, review automatically, and package your tools for the next client.

12

1.7

## Resume and fork

Pick up earlier work by name, and try two ideas from the same starting point.

> **In plain words:** a named session is a bookmark. A fork is a fork in the road: two paths from the same spot. Forks share the files on your computer, but not the conversation.

**Part A · Start a named session:**

```
claude -n day6-angles
```

**BUILD A** Paste, then `/exit`

```
Read campaigns/welcome-series/plan.md and each group's version of email 04 (the Day 6 "Moving soon? Here's 10% off" email). In five short bullets, tell me what each renter group cares about most when deciding to buy. Don't write any files.
```

**Part B · Come back later:**

```
claude --resume day6-angles
```

**BUILD B** Paste, then `/exit`

```
Remind me: what did you say the moving-out group cares about most? One sentence.
```

**Part C · First fork, with its own name:**

```
claude --resume day6-angles --fork-session -n angle-a
```

**BUILD C** Paste, then `/exit`

```
Angle A: write three subject lines for the Day 6 email that focus on peace of mind (the 14-day money-back promise and handing back the keys with confidence). Save them to notes/angle-a.md.
```

**Part D · Second fork, from the same spot:**

```
claude --resume day6-angles --fork-session -n angle-b
```

**BUILD D** Paste, then `/exit`

```
Before you start: do you know anything about an Angle A from our conversation? Answer yes or no first.

Angle B: write three subject lines for the Day 6 email that focus on saving money (10% off with LAUNCH10, and how small $29 is next to a bond). Save them to notes/angle-b.md.
```

**Part E · Start fresh from a summary.**

Run

```
claude --resume day6-angles
```

and ask:

*"Read notes/angle-a.md and notes/angle-b.md. Then write notes/handoff.md in 10 lines or fewer: what each group cares about, both angles, and which one you'd pick and why."*

Then start a brand-new session with

```
claude
```

and ask:

*"Read notes/handoff.md and continue from it: write the final subject line for the Day 6 email."*

**CHECK** New session, then paste

```
Check my session experiment. Don't change any files.

1. Do notes/angle-a.md and notes/angle-b.md each have three subject lines of 50 characters or fewer? Count them.
2. Does either file use the other angle's idea? They were written in separate forks, so they shouldn't.
3. Do any of the subject lines break brand/voice.md, for example fake urgency or a guarantee?
4. Is notes/handoff.md 10 lines or fewer, and does everything it says match the files it describes?

Give me a short table: Test | Pass or Fail | What you saw.
```

```
Then ask me two quick questions, one at a time: when I should resume a session and when I
should start fresh with a summary, and what --fork-session gave me that resuming didn't.
```

**YOU'LL KNOW IT WORKED WHEN**

- The resumed session remembers what it said about the moving-out group.
- Fork B answers "no" about Angle A's conversation. It may notice the `angle-a.md` file, because forks share files.

**IF SOMETHING GOES WRONG**

**"matches 2 sessions":** you forked without `-n`, so the fork kept the original name. Use one of the session ids the error prints, and name forks from now on.

**EXAM NOTE**

To continue a specific earlier investigation, use `--resume <session-name>`. To compare two approaches from one shared starting point, use `fork_session`; on the command line that's `--fork-session`. If the old tool results are stale, start fresh with a structured summary. If files changed since then, name the changed files so Claude re-checks only those.

13

3.6

## Automatic review

Run Claude without chatting, and get results a program can read.

> **In plain words:** instead of chatting, you hand Claude a job sheet and it hands back a filled-in form. The form's shape, called a schema, never changes, so a program can read it.

**BUILD 1 OF 2** Paste in Claude Code

```
Make a review that runs without anyone typing, so it can later run on every pull request.

1. Create review/findings.schema.json: a JSON Schema for an object with a "findings" array.
Each finding has file, line (a number), severity ("high", "medium" or "low"), problem and fix.
All five are required.
2. Create scripts/review.mjs, a small Node script that takes a campaign folder and runs Claude
Code headless, like this:
   claude -p "Review every email in <folder> against brand/voice.md and
.claude/rules/emails.md. Treat any guarantee of a full bond back, anything that reads as legal
advice, fake urgency, a missing unsubscribe link or a missing 'not legal advice' line as high
severity. Report only real problems." --output-format json --json-schema "<the schema file's
text>" --allowedTools "Read,Glob,Grep"
   The output may be one JSON object, or a list whose last item is the result, so handle both.
Take structured_output, save it to review/findings.json, print a one-line summary, and exit
with code 1 if any finding is high severity.
3. To prove it catches problems, write a deliberately bad email at campaigns/test-
bad/emails/01-bad.md. It should break our rules with "We guarantee you'll get your full bond
back", a "Only 3 toolkits left!" fake-scarcity line, legal advice about what to tell VCAT, and
two links. Then run: node scripts/review.mjs campaigns/test-bad
4. Run it on campaigns/welcome-series too, then commit everything.

Tell me what each run found and what exit code it gave.
```

**BUILD 2 OF 2** Same session

```
Improve scripts/review.mjs so a second review doesn't repeat itself.

If review/findings.json already exists for the same folder, include those earlier findings in
the prompt and add: "These are findings from the last review. Report only problems that are
new or still not fixed. Don't repeat ones that have been fixed." To make that work, save the
folder name inside review/findings.json too.

Then test it:
1. Run node scripts/review.mjs campaigns/test-bad and note the findings.
2. Fix only the guarantee claim in campaigns/test-bad/emails/01-bad.md (change it to something
allowed, like "Our checklists help you give yourself the best chance of getting your bond
back").
3. Run the review again on campaigns/test-bad.
```

```
Tell me: did the guarantee finding disappear, and did anything get reported twice? Commit when
done.
```

**CHECK** New session, then paste

```
Test my automatic review. The script rewrites review/findings.json; don't change any other
file.

1. Run node scripts/review.mjs campaigns/test-bad. Did it exit with code 1? Does every finding
in review/findings.json have all five fields from review/findings.schema.json, with severity
high, medium or low?
2. Did it catch the second link and the fake-scarcity line? Quote those findings.
3. Run node scripts/review.mjs campaigns/welcome-series. Did it exit with code 0?
4. Read scripts/review.mjs. Does it handle both output shapes (one object, or a list), and
what does it do if claude fails?

Give me a short table: Test | Pass or Fail | What you saw.
Then ask me two quick questions, one at a time: why a separate headless run does the review
instead of the session that wrote the emails, and why we ask for JSON that matches a schema
instead of plain text.
```

**YOU'LL KNOW IT WORKED WHEN**

- The bad email gives **exit code 1** with findings like the guarantee claim, the fake scarcity, the legal advice and the second link. Your real welcome series gives **exit code 0**.
- After you fix one problem, the next review drops it and repeats nothing.
- Reviews vary a little from run to run. That's why the script only fails on high severity.

### Optional: every pull request

If your studio is on GitHub, ask Claude to add a GitHub Actions workflow that runs `node scripts/review.mjs` on pull requests. It needs your `ANTHROPIC_API_KEY` saved as a repository secret.

**EXAM NOTE**

`-p` runs Claude without waiting for input, which fixes a CI job that hangs. `--output-format json` with `--json-schema` gives results a program can read. CLAUDE.md holds the review rules for CI runs. On a re-review, include the earlier findings and ask for only new or still-unfixed problems.

Not real: `CLAUDE_HEADLESS=true` and `--batch`. In the product, `--json-schema` takes the schema's text, not a file name.

14

Real-world skill 3.4

# Package your studio kit

Plan first, turn the reusable parts into a plugin, and use it for a second business.

> **In plain words:** a plugin is a toolbox you carry from client to client. Each business's brand book stays in that business's own office. If you later start a second product (say, a Sydney version, or a cleaning service), you reuse the toolbox and only write a new brand book.

### Before you start

Press `Shift+Tab` until the bottom of the screen says **plan mode**. Claude will read and plan, but write nothing until you accept.

**BUILD 1 OF 3** Paste in plan mode

```
Plan this first. Don't build anything yet.

I want to package the reusable parts of this studio as a plugin, so I can use them for my next business or client. Plan a new folder next to this one, called studio-kit, with:
- .claude-plugin/plugin.json (name studio-kit, version 0.1.0, a one-line description) and .claude-plugin/marketplace.json listing the plugin with source "./"
- skills/personalize/SKILL.md, copied from our skill, with anything Fullbond-specific (renter groups, bond, toolkit) made general
- hooks/hooks.json, registering our send guard as a PreToolUse hook on mcp__resend__.*, using ${CLAUDE_PLUGIN_ROOT} instead of $CLAUDE_PROJECT_DIR, plus hooks/send-guard.mjs
- agents/email-reviewer.md: a subagent that reviews email files against the project's brand voice and email rules, using only Read, Glob and Grep
- review/findings.schema.json and scripts/review.mjs (with the Fullbond-specific high-severity list moved out, so each business sets its own)
- README.md: what's inside and how to install it

Brand voice, leads, campaigns and the email rule stay in each business's own repo. The README should say why.

Tell me which files you'll create, what you'll change from the originals, and anything that won't work outside this studio.
```

### Then

Read the plan. Find one thing it says won't travel to a new business. Then accept the plan, or paste:

**BUILD 2 OF 3** After you read the plan

```
The plan looks good. Build it. When it's built, run claude plugin validate on studio-kit and
fix anything it reports.
```

**BUILD 3 OF 3** Same session

```
Set up a second, made-up client so I can test studio-kit somewhere new. Create a folder next
to this one called client-two, for a small Melbourne end-of-lease cleaning service called
Spotless Exit, with:
- brand/voice.md: a short voice guide (friendly and practical, no "bond-back guarantee"
claims, sign off with "Sparkling regards,")
- CLAUDE.md: one paragraph about the business, plus the line @brand/voice.md
- leads/leads.csv: 6 made-up people in two groups, house-movers and property-managers, all at
@example.com, with the same columns as our leads file
- leads/CLAUDE.md: our leads folder rules, plus a "Groups" section with one line describing
each group
- .claude/rules/emails.md and .mcp.json, copied from this studio, but change the email rule's
footer and link to Spotless Exit's (use https://spotlessexit.example/book)
- campaigns/first-booking/emails/01-hello.md: one short email that follows the email rules
Then run git init in client-two and commit everything there. Don't install studio-kit; I'll
load it myself.
```

### Then

1. `/exit`, then `cd ../client-two`
2. Start Claude Code with your kit loaded: `claude --plugin-dir ../studio-kit`
3. Approve the resend server when asked. Keep the practice server running.

**CHECK** Inside client-two, then paste

```
Test the studio-kit plugin in this client. Only the personalize skill may write files.

1. Is the studio-kit plugin loaded? List the studio-kit skills and agents you can see.
2. Use the studio-kit personalize skill on campaigns/first-booking. Check that it wrote one
version of the email for each group in leads/leads.csv.
3. Use the resend send-email tool to send campaigns/first-booking/emails/01-hello.md to the
first person in leads/leads.csv. The kit's guard should block it. Quote the block message word
for word, and don't try to get around it.
4. Search the ../studio-kit folder for the words Fullbond, bond toolkit and LAUNCH10. There
should be no matches.

Give me a short table: Test | Pass or Fail | What happened.
Then ask me two quick questions, one at a time: what belongs in the plugin versus in each
business's own repo, and why the guard is a PreToolUse hook instead of a PostToolUse one.
```

**YOU'LL KNOW IT WORKED WHEN**

- `claude plugin validate` says **Validation passed**. A missing-icon warning is fine.
- In client-two you see `studio-kit:personalize` and `studio-kit:email-reviewer`. Personalize uses the cleaning service's own groups.
- The kit's guard blocks a send to a Spotless Exit lead.

**IF SOMETHING GOES WRONG**

**"Fullbond" shows up in client-two:** a rule was copied without being adapted. Rules belong to each business, so ask Claude to update the footer and link for Spotless Exit.

**EXAM NOTE**

Use plan mode when a task is large, has several valid approaches, involves design decisions or touches many files. If the question already says the task is complex, plan from the start; "start direct and switch if it gets complex" is the trap answer. Plugins themselves aren't on the exam, and CLAUDE.md and rules always stay in the repo.

STRETCH · STEP 15

# For fast finishers

A bigger pile of text, and the habits that keep Claude accurate when its memory gets full.

15

5.4

## What testers loved (and what confused them)

Keep track of findings across many files, even after Claude's memory is squeezed.

> **In plain words:** when Claude's memory gets squeezed, a notebook still has everything. So write in the notebook as you go, not at the end. This mirrors Step 8 of your build plan: "Ask 3 friends who rent to try it and tell you one thing that was confusing", just at a bigger scale.

**BUILD 1 OF 3** Paste in Claude Code (in the Fullbond studio)

```
Create practice data: notes/feedback/ with 120 short made-up pieces of feedback from renters
who beta-tested the Fullbond Melbourne Renter Toolkit and the free 10 Bond Mistakes checklist,
in 6 files of 20 each (feedback-1.md to feedback-6.md). Make them varied and realistic: name
specific items (move-in checklist, photo evidence guide, move-out cleaning checklist, 6-week
countdown planner, email templates, moving checklist), mix praise with complaints (download
link went to spam, PDF hard to read on a phone, unclear which kit to buy, wanted a Notion
version, unsure if it applies outside Victoria, cleaning checklist too long), mention a
Melbourne suburb, and give each a star rating. Don't summarise them.
```

### Then

Start a named session: `claude -n feedback-dig`

**BUILD 2 OF 3** Paste in the named session

```
Find the top 5 things testers love and the top 3 complaints across all the feedback in
notes/feedback/.

Use a subagent to read the feedback files, so the raw feedback doesn't fill up this
conversation. Tell the subagent to keep a scratchpad at notes/scratchpad.md and to add to it
after each file, one line per finding: the file, the finding, how many pieces of feedback
mention it, and one short quote. At the end, give me the top 5 and top 3 with how often each
came up, and one idea for how each complaint could change the website or the welcome emails.
```

### Then

Type `/compact`. This squeezes the conversation into a short summary.

**BUILD 3 OF 3** Same session, after `/compact`

```
Which feedback file had the most complaints about the download email going to spam or not
arriving? Quote one. Answer from memory first. Then check notes/scratchpad.md and the files,
and tell me whether your memory was right.
```

**CHECK** New session, then paste

```
Check my scratchpad. Don't change any files.

1. Does notes/scratchpad.md have lines for every file in notes/feedback/?
2. Pick three lines and check each one against the feedback file it names: is the quote real,
and is the count right?

Give me a short table: Test | Pass or Fail | What you found.
Then ask me two quick questions, one at a time: why we write to a scratchpad as we go, and
what /compact does to Claude's memory.
```

**YOU'LL KNOW IT WORKED WHEN**

- Your main session only sends work to a subagent. The raw feedback never fills your chat.
- After `/compact`, memory gets details wrong or can't answer, and the scratchpad can.
- With a thin scratchpad (one line per file), it can't answer either. Write one line per finding.

**EXAM NOTE**

The warning sign is answers that stop matching earlier findings, or that talk about "typical patterns" instead of the specific things found. The fixes are scratchpad files, subagents, summaries between phases, saved state files, and `/compact`. A bigger context window is never the answer.

# Exam map

Where each CCAR-F Claude Code objective is practised in this project.

| Objective | What it covers | Steps |
|---|---|---|
| 3.1 | CLAUDE.md levels, @import, folder CLAUDE.md, /memory | 2, 3, 5 |
| 3.2 | Commands and skills: argument-hint, context: fork, allowed-tools | 4, 8 |
| 3.3 | Rules with paths: patterns | 6 |
| 3.4 | Plan mode, sending work to subagents | 14, 15 |
| 3.5 | Interview first, input/output examples, fixes in one message | 3, 5, 7 |
| 3.6 | Fresh-session review, -p, JSON schema, re-review | 7, 13 |
| 1.7 | Resume by name, fork, fresh start with a summary | 12 |
| 2.4 | .mcp.json, ${VAR} credentials, use what exists first | 1, 9 |
| 2.5 | Glob and Grep | 7 |
| 1.5 | Hooks: the exam names PostToolUse; PreToolUse is what blocks | 10, 14 |
| 5.4 | Big context: scratchpad, subagents, /compact | 15 |

Not practised here: MCP resources (catalogs of data), Edit's fallback when its anchor text isn't unique (Read, then Write), and tracing code through a wrapper module.

# Before you send real emails to real subscribers

This lab never emails real people. Before you switch Resend from practice mode to your real account for the live site, check these first (from Steps 6–8 of the build plan):

- [ ] Every email shows the Fullbond name, your contact address and a working unsubscribe link (Spam Act 2003), with the placeholders filled in.
- [ ] Your domain fullbond.com.au is verified in Resend, so emails come from hello@fullbond.com.au and don't land in spam.
- [ ] The sign-up form has the consent tick box: "Send me renter tips and offers from Fullbond. Unsubscribe any time."
- [ ] Every renting rule mentioned in an email has been checked against Consumer Affairs Victoria.
- [ ] Prices are in AUD and match Payhip exactly; LAUNCH10 is set up in Payhip and really ends when the email says.
- [ ] No reviews or testimonials appear until you have real ones.
- [ ] You signed up with your own email and received all 5 emails in order, not in spam.

# Words you'll see

**Session**
One conversation with Claude Code. `/exit` ends it; running `claude` starts a new one.

**Commit**
A saved snapshot of your files in git. You can always go back to it.

**Plugin**
A package of skills, commands, hooks and connections that you install in one go.

**Skill**
Saved instructions Claude follows when you call it with `/name`, or when the task fits.

**MCP server**
A connection that gives Claude tools from another service, like Resend.

**Hook**
A small script Claude Code runs at a set moment, like just before a tool runs. It can block the action.

**Practice mode**

Resend's tools connected to a pretend email server on your own computer.

**Fresh session**

A new conversation that doesn't remember the old one, which makes it a fair checker.

**Lead magnet**

A free thing (here, the 10 Bond Mistakes checklist) offered in exchange for an email address.

**Welcome series**

Automatic emails sent to each new subscriber over their first days on the list.